

第4章 隠されたバイナリを暴こう

この章で学ぶこと

- パッカー
- 圧縮

パッキングとは

バイナリを**圧縮・暗号化**して、中身が見えにくくする技術のことです。

パッカーと呼ばれる専用のツールを用いてパッキング/アンパッキングは行われます。

ここでは一番有名なパッカーであるUPXを利用していきます。

以下ではパッキングしたバイナリとしていないバイナリの差を見ていきたいと思えます。

`strings` の結果の差を見るのでソースコードの内容はあまり気にしなくてもいいです。

ソースコード：

```

1 fn greet(name: &str) {
2     println!("Hello, {}!", name);
3 }
4
5 fn show_status(code: i32) {
6     match code {
7         0 => println!("[INFO] Process completed successfully."),
8         1 => println!("[WARN] Something went wrong."),
9         2 => println!("[ERROR] Fatal error occurred."),
10        _ => println!("[DEBUG] Unknown status code: {}", code),
11    }
12 }
13
14 fn main() {
15     let version = "1.0.0";
16     let author = "kindai-student";
17     let config_path = "/etc/myapp/config.toml";
18     let log_path = "/var/log/myapp.log";
19
20     println!("≡ MyApp v{} by {} ≡", version, author);
21     println!("Loading config from: {}", config_path);
22     println!("Logging to: {}", log_path);
23
24     greet("World");
25     show_status(0);
26
27     println!("Goodbye!");
28 }

```

出力結果：

```

~/workshop/packing/pack/target/debug master
> ./pack
≡ MyApp v1.0.0 by kindai-student ≡
Loading config from: /etc/myapp/config.toml
Logging to: /var/log/myapp.log
Hello, World!
[INFO] Process completed successfully.
Goodbye!

```

strings結果：

[illegible]

パッキングしてみる

```
upx --force-macos -o <パッキング後のファイル名> <パッキング対象のファイル名>
```

```
~/workshop/packing/pack/target/debug  🐙 master
) upx --force-macos -o packed_pack pack
                                     Ultimate Packer for eXecutables
                                     Copyright (C) 1996 - 2026
UPX 5.1.1           Markus Oberhumer, Laszlo Molnar & John Reiser   Mar 5th 2026

      File size           Ratio           Format           Name
-----
445384 →    196624    44.15%    macho/arm64    packed_pack

Packed 1 file.
```

パッキングされてないやつと全然違う！！！！

```
~/workshop/packing/pack/target/debug master
> strings packed_pack
3UPX!
H__PAGEZER0o
TEXT?@
[?text&0
?stubs0[v

_hrr
e!perpYH_
0Mrin
0gcc_<
cept
0eh_{
frame
GDATA_CONSM@
bo!_!r0
daf7
]0hreaovars0`
0ommz
oLINKEDI
GK"IO`
//usr/lib/dyld
?7:27
%{'L
Hem.B.
7&_!X
gCC[
3Mw;W_
S? ?!
```

```
~/workshop/packing/pack/target/debug master
> ./packed_pack
zsh: killed ./packed_pack
```

UPXでパッキングされたバイナリは残念ながら実行できません...

マルウェア関連で有名すぎて対策されてる（たぶん）

MacOSはちゃんとセキュリティ強いんだと学びましたね...

macOS の XProtect (組み込みマルウェア検知) にやられています。これ、授業的にめちゃくちゃおいしい状況です。

何が起きているか

```
./packed_pack
↓
macOS XProtect が「UPXパック = マルウェアの典型的手口」と判断
↓
SIGKILL で即強制終了
```

Apple は UPX パックされたバイナリをシグネチャベースで検知してブロックします。実際のマルウェアの大半が UPX を使うため、UPX パック自体が検知対象になっています。

本来なら pack を実行した時と同じ結果になります。

想定されていた結果：

```
~/workshop/packing/pack/target/debug master
) ./pack
≡ MyApp v1.0.0 by kindai-student ≡
Loading config from: /etc/myapp/config.toml
Logging to: /var/log/myapp.log
Hello, World!
[INFO] Process completed successfully.
Goodbye!
```

アンパッキング (パックしたものを元に戻す) してみましょう。

```
upx -d <対象のファイル名>
```

```
~/workshop/packing/pack/target/debug/pack_dir  ? master
) upx -d packed_pack
Ultimate Packer for eXecutables
Copyright (C) 1996 - 2026
UPX 5.1.1      Markus Oberhumer, Laszlo Molnar & John Reiser   Mar 5th 2026

File size      Ratio      Format      Name
-----
311296 ←    196624    63.16%    macho/arm64    packed_pack

Unpacked 1 file.

~/workshop/packing/pack/target/debug/pack_dir  ? master
) ls
packed_pack

~/workshop/packing/pack/target/debug/pack_dir  ? master
) ./packed_pack
≡ MyApp v1.0.0 by kindai-student ≡
Loading config from: /etc/myapp/config.toml
Logging to: /var/log/myapp.log
Hello, World!
[INFO] Process completed successfully.
Goodbye!
```

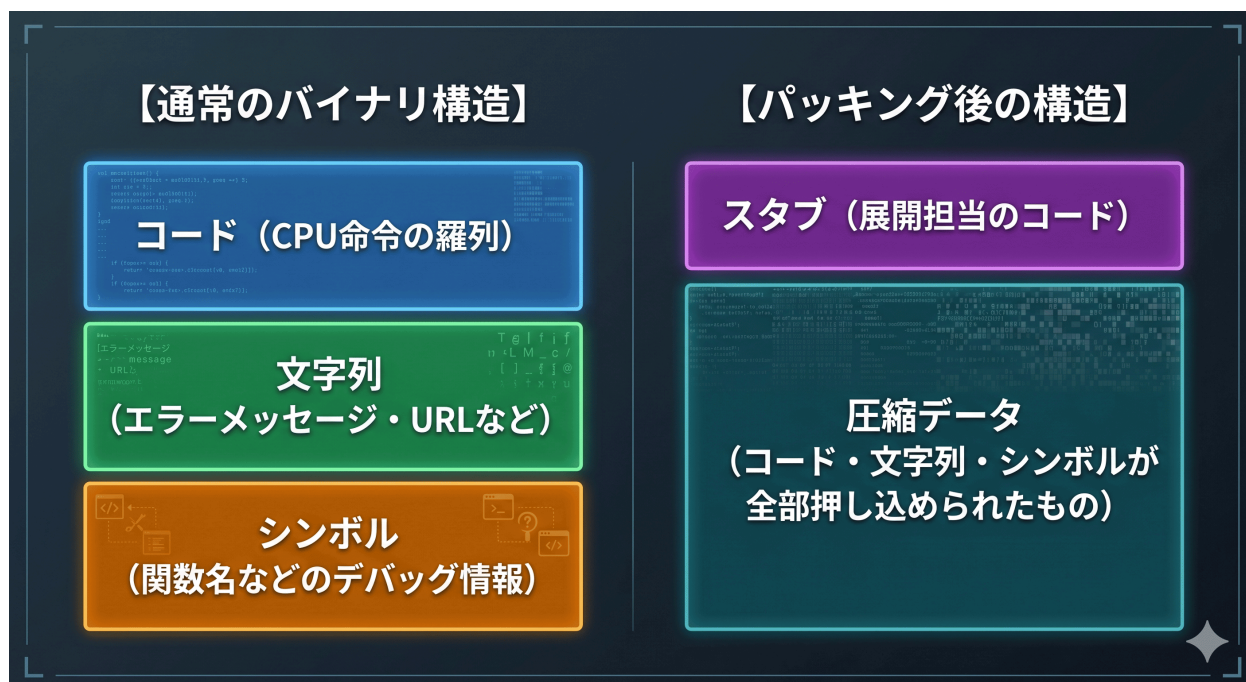
...なんかFile sizeが小さくなって？

アンパックするとUPXがなくなり、実行できるようになります。

実行してみるといいものが見れるかもしれません...（私が配布しているバイナリだけ）

upxの仕組み

スタブ：圧縮された元のコードを展開するためのコード



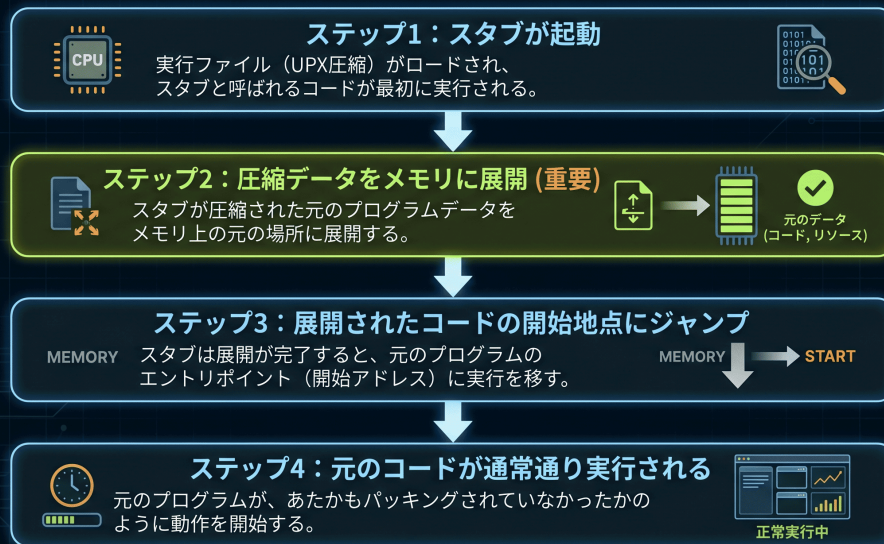
通常のバイナリ：

- 1：コード ← CPU命令の羅列
- 2：文字列 ← エラーメッセージ・URLなど（stringsで見えるやつ）
- 3：シンボル ← 関数名など（デバッグ情報）

パッキング後：

- 1：スタブ ← 展開担当のコード
- 2：圧縮データ ← 上の3つが全部押し込められている

UPXパッキングされたバイナリの実行フロー



1：スタブが起動

2：圧縮データをメモリに展開（元のコードと同じようにする）

3：元のコードを実行するために、展開されたコードの開始地点にジャンプ

4：通常通り実行する

上記がわかりにくい場合の例え

映画の上映に例えると、スタブは**映写技師**のようなものです。

1：圧縮されたフィルムを受け取る

2：上映前に映写機（**メモリ**）にセットして再生できる状態に準備

3：最初のシーンから再生

3：開始観客（**CPU**）はフィルムが元々圧縮されていたことを知らずに普通に映画を観る

by Claude

難しい概念なので、次出会った時に理解するのもありかもしれません。

upx はなぜ重要

アンチウイルスソフトがマルウェアを検知する方法は次の2つに分けられます。

シグネチャ検知：

既知のマルウェアの特徴的なバイト列と照合して一致したら検知

ヒューリスティック検知：

怪しい動作パターンを検知

upxは**シグネチャ検知**を突破してしまうため危険です。

シグネチャ検知は例えば**不審なURL**や**命令**がないかをチェックします。(stringsやxxdみたいなやり方です)

しかし、パッキングされたバイナリは**意味不明**なので、回避されてしまいます。

また、upxは次のような利点があるためよく使われます。

- 1：無料・OSS
- 2：コマンド1つ
- 3：クロスプラットフォーム（Windows/Linux/macOS対応）
- 4：高い圧縮率

しかし、マルウェアで使われすぎてアンチウイルスソフトはパッキングされたバイナリを実行しようとするプロセスをkillしに来ます。悲しいですね...

すべてのマルウェアがupxでパッキングを行っている訳ではありません。

自分の経験では、意味のわからない**パッカー**でパッキングされているものを見て、めんどくさくなって解析をやめた記憶があります。

まとめ

パッキングとはバイナリを**圧縮・暗号化**して、中身が見えにくくする技術です。

よくマルウェアに使われてしまっています。UPXが代表的なツールとしてあります。

パッキング： `upx --force-macos -o <パッキング後のファイル名> <パッキング対象のファイル名>`

アンパッキング： `upx -d <対象のファイル名>`

パッキング前と後で以下のような構成になります。

パッキング前： コード / 文字列 / 関数名
パッキング後： スタブ / ぐちゃぐちゃなデータ

スタブが実行時に圧縮データをメモリ上で展開し、元のコードとして実行する。

ファイルサイズも 16KB → 4KB に圧縮されている。

なんでUPXは危険？

- **シグネチャ検知を回避**できる（strings/xxdで読めないの不審な文字列が見つからない）
- 無料・コマンド1つ・クロスプラットフォームで手軽に使える

以下の課題に取り組んでみてください。

問題8：ディレクトリ4にある通常のバイナリであるnormalとパッキングされたバイナリpacked_normalに対してstringsを行い、違いを確認してみてください。

問題9：ディレクトリ4にあるstrangeバイナリがどんなことをするファイルかを説明してください。実行しても大丈夫なはずです...